

Empirical Analysis of Transformers on Graph Connectivity

Part V: Is It Matrix Powering or DFS?

Edoardo Paccagnella

June 19, 2026

Abstract

The earlier reports read the model’s sharp distance- 3^L wall as evidence that the standard transformer computes connectivity by *matrix powering* (A^{3^L}). This report asks a different question, suggested by the advisor: what if the model is instead learning a *depth-first search* (DFS) — a sequential traversal that explores a component node by node and runs out of budget after a bounded number of steps, rather than a parallel computation bounded by shortest-path *distance*? The two mechanisms make opposite predictions on graphs that are *short but large* — in particular on two dense cliques joined by a single bridge edge — and on whether *density* helps or hurts. This report sets out the hypothesis, the predictions that separate it from matrix powering, and the experiments we will run; results follow in later revisions.

1 Introduction

1.1 What the first four reports established

We start this report from a set of conclusions we now treat as established (each was multi-seed and is documented in Reports I–IV). They are the ground this report builds on, and they are also the facts the DFS hypothesis must remain consistent with.

- **The data lever does not help the standard transformer generalise.** Restricting the training data to within-capacity graphs (diameter $\leq 3^L$), which is *proven* to help the disentangled transformer, does *not* improve out-of-distribution generalisation of a *standard* transformer. Reproducing the paper’s $n = 20$ setup over many seeds, the outcome is **seed-dominated**: whether a run finds the generalising solution is a coin-flip ($\sim 3/8$ seeds in either condition), and the clean “phase transition” the paper shows is, in our hands, a single lucky seed (Report III, Part I). The filter only speeds up convergence (Report II).
- **There is a sharp wall at shortest-path distance $\approx 3^L$ ($= 9$ for our $L = 2$ models), and it is architectural, not a data gap.** The wall moves with depth in a causal depth sweep ($L = 1 \rightarrow d=3$, $L = 2 \rightarrow d=9$, $L = 3 \rightarrow$ the whole range; Report III). It appears even on families that are *in* the training stream and seen $\sim 10^8$ times (2chains, 1cycle at $n = 40$: exactly 1.00 up to $d = 9$, then exactly 0.00; Report IV, Thread 1). So steering the data cannot move it.
- **Density is hard at inference — but so far confounded with being out-of-distribution.** A graph denser than the (sparse) training set fails even when its diameter is tiny (Report IV, Thread 1, density axis). We attributed this to the graph being unlike anything seen in training. The single sharpest instance is the **degree heuristic** on dense 2cliques, which is repaired *only by the right data* (mixed training), not by the read-out or the loss (Report IV, Thread 3).

- **The spectral gap is not a separable axis on natural graphs** (collinear with the diameter, -0.82); a real bottleneck does add difficulty, but only a purpose-built, confound-free probe isolates it (Report IV, Thread 2).
- **A similarity read-out doubles the reach to $2 \cdot 3^L$** (meet-in-the-middle of two 3^L -hop neighbourhoods) and removes the over-connection (cut) error, *but cannot cross a bottleneck* — that limit belongs to the trunk (Report IV, Thread 3).
- **Counting components is trivial** for the same trunk (1-chain vs 2-chain: 100% in 2,000 steps), so the bottleneck of the connectivity task is not sensing global structure but producing correct long-range *per-pair* reachability (Report III).

1.2 The new question: matrix powering or DFS?

Everything above was *interpreted* through one algorithm: matrix powering. An L -layer model that repeatedly multiplies by the adjacency can confirm a connection within 3^L hops, which fits the distance-9 wall exactly. But that wall is also consistent with a *second* algorithm we never separated from it: a **depth-first search**. A transformer cannot recurse without bound in a fixed number of layers, so any DFS it learns must be a *bounded* traversal — it explores a component step by step and *stops after a limited number of steps*, leaving the rest of the component unreachable. The advisor’s hypothesis is that this, not matrix powering, may be what the model is doing: “maybe the algorithm stops after a certain iteration instead of running to the end, so it never finishes learning the graph.”

The distinction matters because the two algorithms are bounded by *different* quantities, and on most graphs those quantities move together (a long graph has both a large distance and many nodes to traverse), which is exactly why the first four reports could not tell them apart. The point of this report is to build graphs where they *disagree*.

2 Two hypotheses, and how to tell them apart

Matrix powering (the reading of Reports I–IV). Connectivity via A^k with self-loops: i and j are connected iff $(A^k)_{ij} > 0$ for k large enough. An L -layer model realises this up to $k = 3^L$. This computation is *parallel* (every pair is resolved at once), *symmetric* (no preferred node or direction), and bounded by the shortest-path *distance* between the two nodes. Crucially, a *denser* graph has *shorter* distances, so under matrix powering density should make connectivity *easier*, not harder.

DFS (the new hypothesis). Connectivity by traversal: start at a node, walk into neighbours, mark everything reachable as one component. A bounded DFS visits at most some fixed number of nodes/edges before its budget runs out. This computation is *sequential*, has a *starting point* (so it can resolve part of a component and not the rest), and is bounded by the *amount of structure to traverse* (number of nodes and edges), *not* by distance. Under DFS a *denser* graph is *harder*: each node has more neighbours, so a fixed budget covers fewer nodes before it is exhausted.

Where they disagree. Table 1 lists the predictions that separate the two. The cleanest single discriminator is a graph that is *short in distance but large in nodes to traverse*: two dense cliques joined by a single edge. There the bridge sits at distance ≤ 3 (trivial for matrix powering, well inside the 9-hop capacity), but reaching the far clique requires traversing one whole dense clique first (expensive for a bounded DFS).

Observation	Matrix powering predicts	DFS predicts
What bounds a correct answer	shortest-path <i>distance</i> ($\leq 3^L$)	<i>nodes/edges</i> traversed
Dense clique + 1 bridge edge	easy (bridge is ≤ 3 hops)	hard (must cross a whole clique)
Effect of density at inference	easier (shorter distances)	harder (more to explore)
Effect of density in training	easier / faster to learn	harder / slower to learn
Error pattern in $\hat{R}$	symmetric, no preferred node	ordered, a start node fully resolved
Asymmetry between two cliques	none	one clique can be right, the other partial

Table 1: Predictions that separate the two hypotheses. The model studied throughout is the standard RoBERTa-faithful transformer with the *linear* read-out of Reports III–IV ($L = 2$, single head, $d_{\text{model}} = 512$), unless stated otherwise.

A note on consistency with prior results. The DFS reading does *not* overturn the 3^L wall: on the sparse, low-degree families of Reports III–IV (chains, cycles, path unions) every node has ≈ 2 neighbours, so “distance” and “nodes traversed along the path” are the same number, and a step-bounded DFS and a distance-bounded matrix power are *indistinguishable* — both wall at 9. That is precisely why those experiments could not separate the two, and why the discriminating experiments below all break that coincidence (dense local structure, or density swept on its own).

3 The experiments we will run

This section records the plan so the logic is legible to a future reader; results will be added as the runs complete. All experiments use the standard RoBERTa-faithful transformer with the linear read-out (the “base” model of Reports III–IV), trained at $n = 20$ and $n = 40$, multi-seed, unless noted.

3.1 Core test: barbell (one bridge) vs. two cliques

The two labels. On n nodes, split into a left half and a right half of $n/2$.

- **Label A — bridged cliques (one component).** Each half is a clique; a *single* edge joins one node of the left clique to one node of the right. The connectivity target R is the all-ones matrix.
- **Label B — two cliques (two components).** The same two cliques, with *no* bridge. R is block-diagonal (left block and right block all-ones, cross-block all-zeros).

The two graphs differ by exactly one edge, and that edge creates a connection at distance ≤ 3 . (This is *not* the existing `generate_barbell_graph`, which joins $\sim n/3$ cliques through a long middle path; we added two new generators, `generate_bridged_cliques_graph` and `generate_split_cliques_graph`, both taking a `clique_size` so we can sweep it.) We run this two ways:

(a) *Eval the base connectivity model.* The RoBERTa, linear read-out model of Reports III–IV, both ER-trained and mixed-trained, at $n = 20$ and $n = 40$, evaluated on the connectivity matrix — what the base model already does. (2cliques is in the mixed training stream; the bridged version is held out, so it is the clean test.)

(b) *A dedicated, trainable classifier.* A graph-level binary classifier (`GraphBinaryClassifier`: the same standard trunk, mean-pool + one logit) trained from scratch on a 50/50 bridged/split stream, to ask whether the architecture can be *taught* to tell the two labels apart at all, and how that depends on the clique size. We train one model with the clique size drawn at random per sample and read accuracy *by* clique size (flat = matrix powering; falling at large $c = a$ bounded-traversal limit), plus a few fixed- c runs to time convergence against density.

What we measure.

1. **Can the base model tell A from B?** Both as the full connectivity matrix (does $\hat{R}$ contain the cross-block connection for A and not for B?) and, as a cleaner readout, as a graph-level discrimination of the two labels. Matrix powering $\Rightarrow$ easy at any clique size; DFS $\Rightarrow$ fails, and fails worse as the cliques grow denser/larger.
2. **Where in $\hat{R}$ is the error (the “stopped exploring” signature)?** For label A, split the matrix into the left-clique block, the right-clique block, and the cross block, and report exact-match per block. The DFS signature is an *asymmetry*: the clique containing the (implicit) start node is resolved (block ≈ 1.00) while the cross block — the far clique’s connection to the near one — is only partially right (≈ 0.5), because the traversal crossed the bridge but ran out of budget inside the far clique. Matrix powering predicts no such asymmetry.
3. **Scaling with clique size.** Sweep $n/2 \in \{\dots\}$ (clique size). Under matrix powering the cross-block accuracy is flat in clique size (distance stays ≤ 3); under DFS it falls as the cliques grow (more to traverse before reaching / finishing the bridge).

3.2 Re-reading the existing barbell results for the same signature

Report IV already evaluated the (path-bridge) barbell per distance and found the mixed model collapses at the bridge while getting within-clique pairs right. We will re-open those predictions at the *block* level (left clique / right clique / cross), to check for the same asymmetry — whether one side is resolved and the other is not — without training anything new. This is the cheap confirmation: the runs exist, we only add the per-block breakdown.

3.3 Density in optimisation, not just inference

Reports II–IV studied density (and diameter) at *inference*. The DFS hypothesis makes a prediction about *training*: if the model learns a traversal whose cost grows with the amount of structure, then *denser* training graphs should make optimisation *harder* / *slower*, whereas matrix powering (shorter distances when dense) predicts the opposite. We will sweep the training density (e.g. ER edge probability, or mean degree) at fixed n and measure convergence speed and final in-distribution accuracy, and the OOD transfer of each. We separate this from the diameter sweep of Report II (which varied distance, and found tighter = faster): here we hold the regime and vary *density* on its own, the quantity a bounded traversal (DFS or BFS) is sensitive to and matrix powering is not.

3.4 Reference algorithms: matrix power, bounded DFS, and bounded BFS

To make “looks like a bounded traversal” a measurement rather than a story, we implement *three* reference connectivity-matrix predictors (`dfs_oracle.py`) and compare the *transformer’s error pattern* to each. Crucially we keep the two visit-bounded traversals *separate*, because — as the next paragraph shows — they make *opposite* predictions on this graph, so they are not interchangeable and the experiment can tell them apart.

- a **matrix-power oracle**: $\mathbf{1}[(A+I)^{3^L} > 0]$, the clean distance-bounded predictor (the cut oracle of Report IV). On the bridged cliques it connects the two sides at *any* clique size (the bridge is ≤ 3 hops), so its cross-block curve is flat at 1.
- a **bounded-DFS oracle**: from each start node run a *depth*-first search and stop after visiting a fixed **budget** of nodes, calling them connected.
- a **bounded-BFS oracle**: from each start node run a *breadth*-first search (a ball of the budget nearest nodes) and call them connected.

Both visit-bounded oracles are bounded by the *number of nodes visited*, the property that distinguishes them from the distance-bounded matrix power; what differs is the *order* of exploration, and on two dense cliques on one bridge that order changes everything.

Depth-first dives, breadth-first gets stuck — and that opposite trend is what separates them. The advisor’s word was “DFS”, loosely “a traversal that stops early”; making it precise forces the split above, and on this graph the two halves behave *oppositely*. A bounded *DFS dives*: from a node in the near clique it reaches the bridge endpoint within a couple of steps and crosses to the far clique almost immediately, so it does *not* get stuck in the near clique — its cross-block rate is high, and *rises* as the budget (and clique size) grows, because a larger budget reaches deeper into the far clique. A bounded *BFS* instead *gets stuck*: it first fills the whole near clique (all $c - 1$ neighbours at distance 1) and only crosses the bridge once the budget exceeds the near-clique size c — so its cross-block rate is low and *falls* with the clique size, however few hops away the far clique is. This opposite trend (DFS up, BFS down) is exactly the lever the clique-size sweep uses to decide which one the model resembles; the directional asymmetry of the single-start DFS (one clique resolved, the other partial) is a second lever.

We score, per family, which oracle the model’s mistakes resemble — agreement on the *cross-block* entries where the matrix-power and traversal oracles disagree, not just overall accuracy — and how that agreement moves with the clique size and with the budget. If the model’s cross-block accuracy falls with the clique size, that already rules the depth-first oracle out (it *rises*); whether it then tracks the breadth-first oracle at a small budget, or stays flat at 1 like matrix power, is what decides between a visit-bounded traversal and matrix powering. (As Section 4.2 reports, the model falls, has no asymmetry, and matches the breadth-first oracle — so from there on we carry matrix power vs. bounded-BFS.)

Implementation (this session). New code: the two generators in `data.py`; `dfs_oracle.py` (matrix-power, bounded-BFS, bounded-DFS); `eval_bridged_cliques.py` (discrimination, per-block, asymmetry, clique-size sweep, oracle agreement; eval-only) with `scripts/eval_bridged_cliques.sbatch` over the base checkpoints; the dedicated classifier `experiments2/train_bridged_classifier.py` with `scripts/bridged_classifier.sbatch`; the density-in-training sweep reuses `train_families_n20.py` (it already takes `-p/-n_nodes`) via `scripts/density_sweep.sbatch`; plotting in `plot_bridged_cliques.py` and `plot_density_sweep.py`. Outputs land under `runs/report5/`.

4 Results

The runs of Section 3 are completing in order. We report three: the dedicated trainable classifier (Section 3.1, part (b)), in Section 4.1; the decisive one — the base connectivity model evaluated on the bridged cliques (Section 3.1, part (a), with the per-block, clique-size-sweep and oracle-agreement reads), in Section 4.2; and the density-in-optimisation sweep (Section 3.3), in Section 4.3. Three further experiments are queued or launched and reported as stubs here: the oracle agreement across the natural families (the capstone, Section ??); the bridged probe under the similarity read-out, asking whether the node budget scales with the read-out (Section ??); and the capacity-vs-data-prior pair — training with bridged cliques in the stream, plus a depth sweep (Section ??). The final matrix-powering-vs-DFS verdict (Section ??) is written once these land.

4.1 The dedicated classifier: the bridge is detectable at every clique size

Setup recap. We train the binary classifier of Section 3.1(b) (the standard minimal trunk — $L = 2$, single head, $d_{\text{model}} = 512$, normalised-ReLU — with mean-pool + one logit) from scratch

at $n = 20, 200,000$ steps. The data is an *online stream*: there is no fixed training set, each graph is freshly generated at every step, with the bridged/split label drawn 50/50. A graph of “clique size c ” is two cliques of c nodes (so $2c$ of the $n = 20$ nodes are used, the remaining $20 - 2c$ left isolated), bridged or not; c can range over $\{2, 3, \dots, 10\}$ ($c = 10$ fills the whole canvas, no padding).

The two **regimes** differ only in how c is chosen for each graph of the stream, and they ask two different questions:

- **Random c (the discriminator).** For *every* training graph, c is drawn uniformly from $\{2, \dots, 10\}$, so a *single* model sees all clique sizes mixed together (each $\approx 1/9$ of the stream). At evaluation we read accuracy *separately per clique size*: does the *same* model get worse as the cliques grow? Flat in c is the matrix-powering reading, falling at large c is the bounded-traversal (DFS-like) reading. Four seeds.
- **Fixed c (convergence vs. density).** The whole stream uses a single clique size; we train a separate model on $c = 3$, on $c = 6$ and on $c = 10$ (two seeds each). Here the question is about *optimisation*, not capacity: is a denser clique slower or harder to learn? We compare convergence speed and loss across the three densities.

The balanced test set has 200 graphs per label per clique size (covering $c = 2, \dots, 10$ in the random regime, the single c in the fixed regime).

Final accuracy is flat at 1.0 across every clique size. By the end of training all four random- c seeds reach ≈ 1.00 accuracy at *every* clique size $c = 2, \dots, 10$ (the largest pair of cliques fills the whole $n = 20$ canvas); the worst single cell over all seeds and sizes is 0.995. The fixed- c runs tell the same story: $c = 3, 6$ and 10 all reach 1.00, and in fact already at the first evaluation (2,000 steps). So there is *no permanent accuracy wall* in the clique size: the architecture can be taught to separate the two labels however large the cliques get, which is the matrix-powering-consistent outcome and rules out the strongest form of the bounded-traversal hypothesis (that a bridge behind a large clique is simply *undetectable*). (This classifier cannot tell bounded-DFS from bounded-BFS — both predict the same difficulty ordering below; that separation waits for the base model, Section 4.2.)

But the bounded-traversal difficulty ordering does show up — in the learning dynamics. The flat final curve hides a transient that is exactly the shape a bounded traversal predicts. Figure 1 plots, for each seed, the accuracy of every clique size as a function of the training step (one line per size, colour = size): the small cliques shoot up to 1.0 almost immediately, the larger ones only later, so the curves fan out by size before all converging — the larger, denser cliques are the *last* to be mastered. Read as a snapshot early in training (Table 2, step 2,000): seed 1000 is 1.00 up to $c = 6$ but only 0.65 at $c = 8$ and 0.55 ($\approx$ chance) at $c = 10$; seed 3000 is milder (0.87 at $c = 10$), seed 4000 milder still, seed 2000 already flat. This is a bounded-traversal-like ordering that the model overcomes with more steps, not a hard limit. The fixed- c runs corroborate the optimisation side (Figure 3): although accuracy reaches 1.0 at every c , the early (2,000-step) training *loss* rises with the clique size (≈ 0.05 at $c = 3$, ≈ 0.17 at $c = 6$, ≈ 0.35 at $c = 10$), so a denser clique is genuinely a bit harder to fit — the “density in optimisation” effect a bounded traversal predicts — just not hard enough to delay reaching 100%.

seed	$c=2$	$c=3$	$c=4$	$c=5$	$c=6$	$c=7$	$c=8$	$c=9$	$c=10$
1000	1.00	1.00	1.00	0.97	0.99	0.91	0.65	0.59	0.55
2000	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
3000	1.00	1.00	0.99	0.99	0.99	0.93	0.95	0.92	0.87
4000	1.00	1.00	1.00	1.00	0.99	1.00	0.98	0.99	0.97

Table 2: *Model*: the bridged-vs-split binary classifier — minimal/A.1-style trunk ($L = 2$, single head, $d_{\text{model}} = 512$, normalised-ReLU) with mean-pool + one logit — trained from scratch at $n = 20$ (200,000 steps) on the online 50/50 bridged/split stream with the clique size drawn uniformly per sample (*random-c* regime), four seeds. *Test set*: balanced, 200 graphs per label per clique size $c = 2, \dots, 10$. Classification accuracy **by clique size, early in training (step 2,000)**; bold marks the cells near chance. The larger cliques are learned last (seed-dependently); by the end of training every cell is ≈ 1.00 (the trajectories all converge, Figure 1), so this is a transient ordering, not a wall.

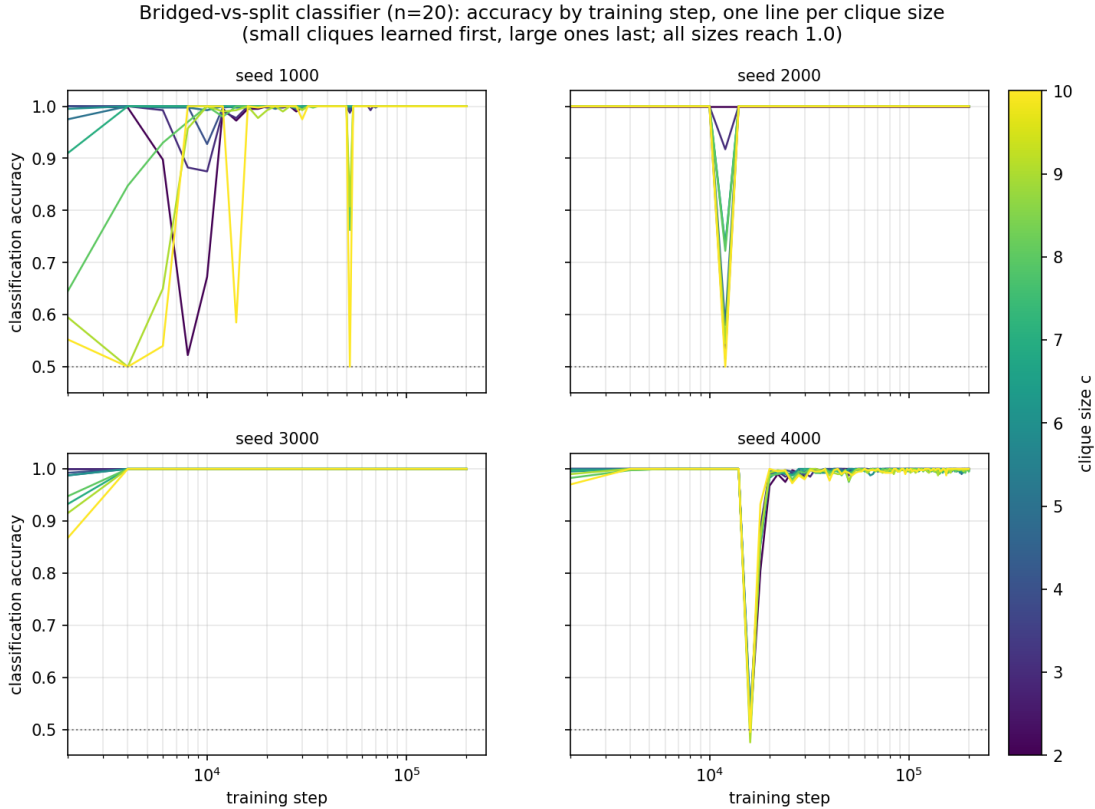


Figure 1: *Model*: the bridged-vs-split classifier (minimal trunk $L = 2$, single head, $d_{\text{model}} = 512$, mean-pool + one logit), trained from scratch at $n = 20$ on the online 50/50 bridged/split stream, *random-c* regime (clique size drawn uniformly $\{2, \dots, 10\}$ per sample), one panel per seed. *Test set*: balanced, 200 graphs per label per clique size. **Accuracy vs training step (log axis), one line per clique size** (colour = clique size c). Small cliques (dark) reach 1.0 almost at once, the larger ones (yellow) later, and every size converges to 1.0 — the difficulty ordering is in *when* a size is learned, not in whether it is. (At $n = 20$, $c \leq 10$, the spread is small and partly hidden by mid-run noise; it is much clearer at $n = 40$, Figure 2.)

Pushing the range: $n = 40$, cliques up to $c = 20$. At $n = 20$ the largest clique is $c = 10$, so the “flat at 1.0” conclusion is only tested that far; a bounded-traversal limit, if real, might only bite at larger cliques. We therefore re-ran the whole classifier at $n = 40$, where the clique

size reaches $c = 20$ (two 20-cliques fill the entire canvas), with fixed sizes $c \in \{5, 10, 15, 20\}$. The conclusion is the *same, only stronger*. The final accuracy is **exactly 1.0 at every clique size** $c = 2, \dots, 20$, **for all four seeds** (no dips at all, cleaner than $n = 20$): doubling the range does not produce a wall, so the bridge stays detectable behind even a 20-node dense clique — the matrix-powering-consistent reading, and a sharper refutation of the strong bounded-traversal hypothesis (the “undetectable” form).

The bounded-traversal cost reappears — as an optimisation cost, more clearly than at $n = 20$. With the larger range the difficulty ordering is now unmistakable. The per-size trajectories (Figure 2) fan out widely: small cliques are learned almost at once, the largest only after tens of thousands of steps. As a snapshot at step 2,000 (Table 3) accuracy falls *monotonically* with the clique size: seed 4000 is 1.00 at $c = 2$, 0.77 at $c = 8$ and flat at chance (≈ 0.50) from $c \geq 11$, with seed 3000 similar and seeds 1000/2000 slower still — the larger the clique, the later it is mastered. The fixed- c runs make the optimisation side explicit and sharp (Figure 3): the step-2,000 training loss climbs from ≈ 0.1 at $c = 5$ to ≈ 0.69 ($= \ln 2$, i.e. *still at chance*) at $c = 15$ and $c = 20$, and the step at which accuracy first reaches 0.99 grows from 2,000 (at $c = 5, 10$) to 4,000–6,000 (at $c = 15, 20$). So a denser/larger clique is genuinely harder and slower to learn — exactly the “density in optimisation” prediction of a bounded traversal (DFS or BFS alike) — but, given enough steps, never an accuracy ceiling. The bounded-traversal difficulty shows up as a *training cost*, not a capability limit. (At $n = 20$, where $c \leq 10$, this was barely visible: every fixed- c run was already at 1.0 by the first evaluation.)

seed	$c2$	$c5$	$c8$	$c11$	$c14$	$c17$	$c20$
1000	0.62	0.69	0.67	0.66	0.59	0.56	0.49
2000	0.50	0.55	0.47	0.50	0.52	0.50	0.50
3000	0.98	0.76	0.51	0.51	0.50	0.50	0.50
4000	1.00	0.88	0.77	0.52	0.52	0.50	0.50

Table 3: *Model*: the bridged-vs-split classifier (minimal trunk $L = 2$, single head, $d_{\text{model}} = 512$, mean-pool + one logit), trained from scratch at $n = 40$ on the online 50/50 bridged/split stream, *random- c* regime (clique size drawn uniformly $\{2, \dots, 20\}$ per sample), four seeds. *Test set*: balanced, 200 graphs per label per clique size. Classification accuracy **by clique size, early in training (step 2,000)** (cols thinned to $c = 2, 5, \dots, 20$); bold marks cells at chance (≈ 0.50). The drop with clique size is monotone and much deeper than at $n = 20$ (Table 2); by the end of training every cell is exactly 1.00 for every seed.

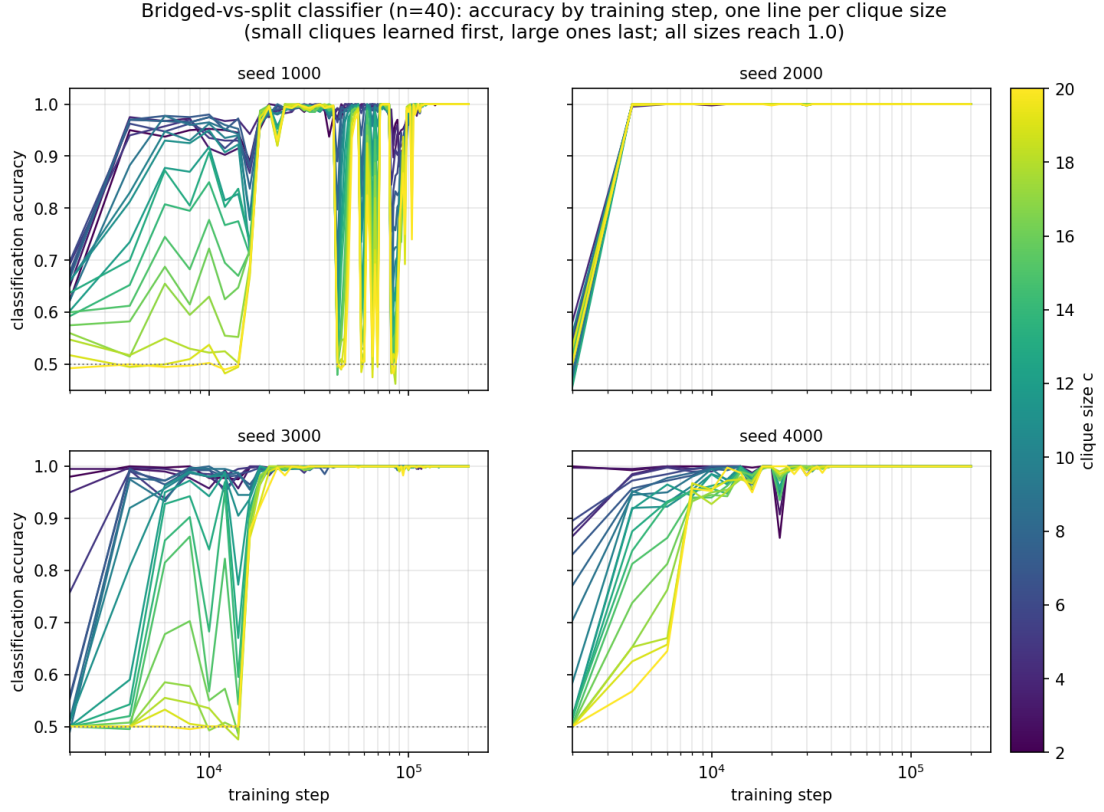


Figure 2: *Model*: the bridged-vs-split classifier (minimal trunk $L = 2$, single head, $d_{\text{model}} = 512$, mean-pool + one logit), trained from scratch at $n = 40$ on the online 50/50 bridged/split stream, *random-c* regime (clique size drawn uniformly $\{2, \dots, 20\}$ per sample), one panel per seed. *Test set*: balanced, 200 graphs per label per clique size. Same layout as Figure 1: accuracy vs training step (log axis), one line per clique size (colour = size). The fan is wide: small cliques (dark) reach 1.0 almost at once, the largest (yellow) only after tens of thousands of steps, but *every* size converges to 1.0. The ordering is monotone in c and far clearer than at $n = 20$.

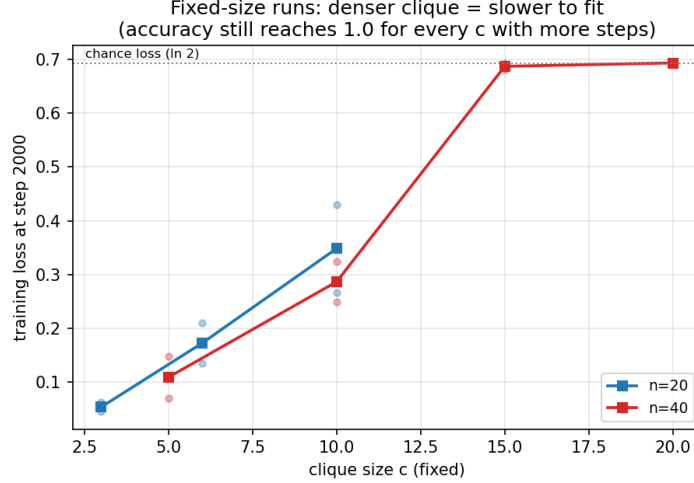


Figure 3: *Model*: the bridged-vs-split classifier (minimal trunk $L = 2$, single head, $d_{\text{model}} = 512$, mean-pool + one logit), but a *separate* model trained from scratch on each single *fixed* clique size (*fixed-c* regime: the online 50/50 bridged/split stream uses one c), $n = 20$ ($c \in \{3, 6, 10\}$) and $n = 40$ ($c \in \{5, 10, 15, 20\}$), two seeds each. *Plotted*: the **training** loss (on the stream, not a test metric) at step 2,000 vs the clique size (markers = the two seeds, line = mean). The loss rises with the clique size and saturates at the chance value $\ln 2$ (dotted) for the largest $n = 40$ cliques — a denser clique is genuinely slower to fit (“density in optimisation”). The held-out accuracy nonetheless reaches 1.0 at every size with more steps, so this is a convergence-speed effect, not a capability wall.

Two caveats, stated plainly. First, training is *noisy* at both sizes: some seeds suffer transient mid-run collapses to chance on the hardest cliques before settling (the step at which all clique sizes stabilise above 0.99 ranges from $\sim 4,000$ to $\sim 116,000$ across the $n = 40$ seeds), so the clique-size ordering is a tendency of the early dynamics, not a clean monotone law — the familiar seed lottery of Reports III–IV, here at the level of which clique sizes are mastered when. Second, and more important for the headline question: this classifier is a *weak* discriminator of matrix powering vs. a bounded traversal. The two labels differ by exactly one edge, and that edge leaves a purely *local* footprint — the two bridge endpoints have degree c instead of $c - 1$, and the total edge count differs by one — so the labels are separable without ever propagating connectivity across the cliques. A flat 1.0 therefore shows the bridge is *detectable* at any clique size; it does *not* show that the model computes the full connectivity matrix by matrix powering. That stronger question — whether the base connectivity model actually fills $\hat{R}$ correctly across the bridge, and whether its errors carry the bounded-traversal signature — is what the base-model eval (Section 3.1(a)) and the oracle comparison (Section 3.4) decide. We turn to it now, and it gives a sharper answer than the classifier.

4.2 The base model: the single bridge is not propagated across a large clique

Setup recap. This is the decisive test (Section 3.1(a)). We take the base connectivity model of Reports III–IV — the RoBERTa-faithful transformer with the *linear* read-out ($L = 2$, single head, $d_{\text{model}} = 512$), the model that emits the full $n \times n$ connectivity matrix $\hat{R}$ — and feed it the bridged and split cliques *without any further training* (eval-only). We use both the ER-trained and the mixed-trained checkpoints, at $n = 20$ and $n = 40$, four seeds each (16 checkpoints). The **mixed** model is the clean test: dense cliques (2cliques, clique-blocks) are in its training stream, so it is *not* out-of-distribution on the cliques themselves; the **ER** model has only seen sparse graphs, so on dense cliques it is in the degree-heuristic OOD regime of Report IV (within-clique

accuracy already < 1) and we read it only as a confounded cross-check, not as evidence on its own (Report IV, Thread 1). For each bridged graph we split $\hat{R}$ into the left-clique block, the right-clique block and the cross block, and we sweep the clique size c (so $2c$ nodes are used, the rest left isolated). The bridge always sits at distance ≤ 3 , *at every c* — so matrix powering must score the cross block 1.0 for all c , while a visit-bounded traversal must lose it once one clique alone exhausts the budget.

What “cross-block pair accuracy” means (and why it is the quantity that matters). The model outputs an $n \times n$ matrix $\hat{R}$; entry $\hat{R}_{ij}$ is its guess for “are nodes i and j in the same component?” A *pair* is one such (i, j) entry, and *pair accuracy* on a set of pairs is just the fraction of those entries the model gets right (predicted 0/1 equals the true 0/1). We group the pairs into three blocks: *within-A* (both nodes in the left clique), *within-B* (both in the right clique), and *cross* (one node in each clique). On a *bridged* graph the two cliques are one component, so the true label of *every* cross pair is 1 (connected); cross-block accuracy is therefore the fraction of A–B pairs the model correctly calls connected, and ≈ 0.00 means it calls essentially all of them *disconnected*. The key thing to keep separate: the bridge is a single *edge*, and it lives in the *input* adjacency A (the model can see it); the cross *block* lives in the *output* $\hat{R}$ and has c^2 entries. Filling it correctly does not mean copying one edge — it means *propagating* that one edge into “and therefore all c^2 cross pairs are connected.” Seeing the edge and propagating it are different operations, and that distinction is the whole story below.

Within each clique: perfect. Across the bridge: collapses with clique size. The mixed model resolves each clique *perfectly* and reports two components on the split graph *perfectly* — within-A, within-B and the whole split matrix are all exactly 1.00, every seed, both n (Table 4). So the failure is *not* that dense cliques are hard (they are in-distribution; the model gets them right). The only thing the mixed model misses is the *cross* block of the bridged graph: at full clique size ($c = 10$ at $n = 20$, $c = 20$ at $n = 40$) the cross-block accuracy is ≈ 0.00 — it sees the two bridged cliques as *two* components, exactly as if the bridge were not there. And the collapse is graded in the clique size (Figure 4, Table 5): cross accuracy is flat at 1.0 for small cliques and falls to 0 as the cliques grow ($n = 20$: 1.0 up to $c = 6$, then 0.27 at $c = 7$, 0.01 at $c = 10$; $n = 40$: 1.0 up to $c = 6$, 0.74 at $c = 7$, 0.27 at $c = 8$, 0 from $c \simeq 14$). The matrix-power oracle, on the same graphs, is flat at 1.0 for every c (the bridge is ≤ 3 hops). **The model departs from matrix powering exactly where a bounded traversal would: once the near clique alone is too big to “get through” before reaching the far one.**

The cutoff is an absolute node budget, not a fraction of the graph. The collapse happens at roughly the *same absolute clique size* at $n = 20$ (knee at $c \approx 6$ –7) and $n = 40$ (knee at $c \approx 7$ –8), even though $c = 7$ is 70% of the $n = 20$ canvas but only 35% of the $n = 40$ one. A traversal that can cover about 6–7 nodes before running out of budget explains both. That is precisely the quantity the DFS hypothesis says bounds the model — the *number of nodes traversed* — and *not* the shortest-path distance, which is ≤ 3 at every c and would predict no collapse at all.

The two visit-bounded references make opposite predictions here — and bounded-DFS is rejected twice over. This is the experiment that separates a bounded *depth*-first search from a bounded *breadth*-first one, and the data picks the second on two independent grounds. (i) *Trend with clique size* (Figure 4, the two extra oracle curves). A bounded DFS *dives*: from a node in the near clique it reaches the bridge in a step or two and crosses, so a *bigger* budget lets it reach *deeper* into the far clique — its cross-block accuracy *rises* with the clique size (0.19 $\rightarrow$ 0.62 at $n = 20$, $\rightarrow$ 0.69 at $n = 40$, orange). The model does the *opposite*: it *falls* from 1 to 0. A bounded BFS fills the near clique first and only crosses once the budget

exceeds it, so its cross accuracy is low and falling (red) — the same direction as the model. So the model is *not* a depth-first search; its trend matches the breadth-first ball. (ii) *Asymmetry*. A single-start DFS also predicts a *directional* signature (Table 1): the start clique fully resolved, the far one only partial, $\hat{R}_{ij} \neq \hat{R}_{ji}$. We see none — both cliques are resolved equally (1.00 vs 1.00), both cross directions fail equally ($\hat{R}_{AB} \approx \hat{R}_{BA} \approx 0$), and the directional-disagreement rate is ≈ 0 . The reason is structural: a transformer emits all of $\hat{R}$ *in parallel*, so there is no single start node and no preferred direction. Both checks point to the same reference, the *visit-bounded BFS ball* we anticipated in Section 3.4: every node resolves the budget-many nearest nodes (its own clique, exactly right) and drops the bridge into the far clique once the near clique alone fills the budget. We therefore set the depth-first reading aside and carry the contrast *matrix powering vs bounded-BFS* forward (it is also the contrast the natural-families capstone uses).

How the oracle comparison works (and why the oracles are not “always right”).

The two oracles are *not* scored for correctness — they are two *reference algorithms*, and we ask which one the model’s answers (mistakes included) look like. One of them, the *matrix-power oracle* $\mathbf{1}[(A + I)^9 > 0]$, does happen to be the true connectivity, so on these graphs it is right everywhere (cross block = 1). The other, the *visit-bounded BFS* with a fixed budget, is *deliberately* an early-stopping algorithm: it explores a ball of the nearest budget-many nodes and calls only those connected, so on the bridged cliques it gets the cross block *wrong* on purpose (it never leaves the near clique). They are useful precisely because they *disagree*, and exactly on the cross block. So the comparison is: take the model’s 0/1 for each pair and ask what fraction matches each oracle’s 0/1. “Overall agreement with the matrix-power oracle is only 0.48” means the model’s matrix matches the *true* connectivity on only 48% of pairs — which is just the within-block pairs; it disagrees with the truth on essentially the whole cross block. The visit-bounded BFS, on the other hand, the model matches almost perfectly: at $n = 20$, with the budget set to one clique ($b = c = 10$), the model agrees with the BFS oracle on 99% of all pairs, and on the contested cross entries — the pairs where the two oracles split — it sides with the BFS oracle $\approx 99.6\%$ of the time and with matrix powering 0.4% (Figure 5; identical across the four seeds, and the same direction at $n = 40$, where it reaches $\sim 79\%$ BFS on the contested entries at $b = 10$ and keeps climbing toward the clique size). One honest refinement, visible in the clique-size sweep (Figure 4) rather than in this single- c scan: at *small* cliques the model is genuinely *matrix power* (cross block = 1.0, where the BFS oracle is already near 0), not BFS. The fuller statement is therefore that the model computes connectivity correctly *up to a node budget* of $\sim 6\text{--}7$ — where it equals matrix power — and collapses to the stuck-BFS floor beyond it. That is exactly a *visit-bounded* computation: matrix power within the budget, a stalled traversal past it. The agreement numbers above are measured at the full clique size, where the budget is already exceeded and the model has settled onto the BFS branch.

The ER model is the confounded cross-check (do not read it alone). The ER curves (Figure 4, bottom) fall with c too, but for a mixture of reasons we cannot disentangle (Table 6): the ER model is already wrong *within* the cliques (degree-heuristic OOD, within-clique accuracy 0.0–1.0 depending on the seed), and it is far noisier seed-to-seed (one $n=40$ seed collapses to 0 everywhere; another rises back toward 1 at large c only because the degree heuristic calls everything dense “connected” — it then *also* wrongly calls the *split* cliques connected, so its discrimination stays at chance). A falling ER curve is therefore not clean evidence of bounded traversal — density-OOD and bridge-distance grow together (Report IV, error noted in our handoff). This is exactly why the mixed model is the test that counts, and there the within-clique blocks are perfect, so the cross collapse is attributable to the bridge alone.

Why this does not contradict the classifier (cross block ≈ 0 here, accuracy 1.0 there).

This is the natural objection, so it is worth spelling out: in Section 4.1 the classifier reached 1.0

at every clique size, and here the same kind of model scores 0.00 on the same bridged graph. There is no contradiction, because the two models are asked *different questions*. The classifier emits *one bit* — “is there a bridge, yes or no?” — and that bit is readable from a purely *local* fingerprint of the bridge edge: the two endpoints have degree c instead of $c - 1$, and the graph has one more edge in total. You can tell bridged from split by counting, without ever working out who is connected to whom. (This is the weak-discriminator caveat of Section 4.1: detecting the edge is easy.) The base model is asked the *harder* question — fill in all c^2 cross pairs of $\hat{R}$ — and that requires *propagating* the single bridge edge into “therefore every node on the left is connected to every node on the right.” It is this propagation, not the detection, that fails: the model plainly *sees* the bridge edge (it is right there in the input A , and the classifier proves it is usable), but it does *not* carry that connection across a clique of more than ~ 6 nodes when it builds the connectivity matrix. So the user’s intuition — “can’t it just read off the 1 at the bridge position?” — is exactly what the classifier does; the base model’s job is the opposite of reading off one entry, it is spreading that one entry over the whole cross block, and that is where the bounded traversal runs out. On the decisive read — a model that already handles the dense cliques, failing the bridge in a graded, absolute-node-budget way that tracks a visit-bounded traversal and parts from the matrix-power oracle — the base-model eval comes down on the side of a *bounded-traversal* mechanism, not distance-bounded matrix powering.

		bridged graph (one component)			split graph
n	seed	within-A	within-B	cross	exact-match
$n = 20$, clique size $c = 10$ (fills the canvas)					
20	1000	1.00	1.00	0.00	1.00
20	2000	1.00	1.00	0.01	1.00
20	3000	1.00	1.00	0.01	1.00
20	4000	1.00	1.00	0.01	1.00
$n = 40$, clique size $c = 20$ (fills the canvas)					
40	1000	1.00	1.00	0.00	1.00
40	2000	1.00	1.00	0.00	1.00
40	3000	1.00	1.00	0.00	1.00
40	4000	1.00	1.00	0.00	1.00

Table 4: *Model*: the base connectivity transformer — RoBERTa-faithful trunk, *linear* read-out ($L = 2$, single head, $d_{\text{model}} = 512$), emitting the full $n \times n$ matrix $\hat{R}$ — **mixed**-trained (the 9-family online stream of Reports III–IV, which includes dense cliques), four seeds, $n = 20$ and $n = 40$; *eval-only*, no further training. *Test set*: bridged cliques and split cliques at the full clique size (2,000 graphs each). *Metric*: per-block pair accuracy (within each clique, and the cross block $A \leftrightarrow B$) on the bridged graph, and exact-match on the split graph; per seed. The within-clique blocks and the entire split matrix are perfect; only the *cross* block (the single bridge) is missed (bold), at every seed. The two cross directions agree (no asymmetry), and the discrimination accuracy is 0.50 (the model calls bridged and split alike — both “two components”). The ER model is omitted here (within-clique accuracy < 1 , confounded; see text).

Bridged cliques: does the model carry the single bridge across the cliques as they grow?
model FALLS like a stuck bounded-BFS, the OPPOSITE of bounded-DFS (which rises); matrix power stays flat at 1

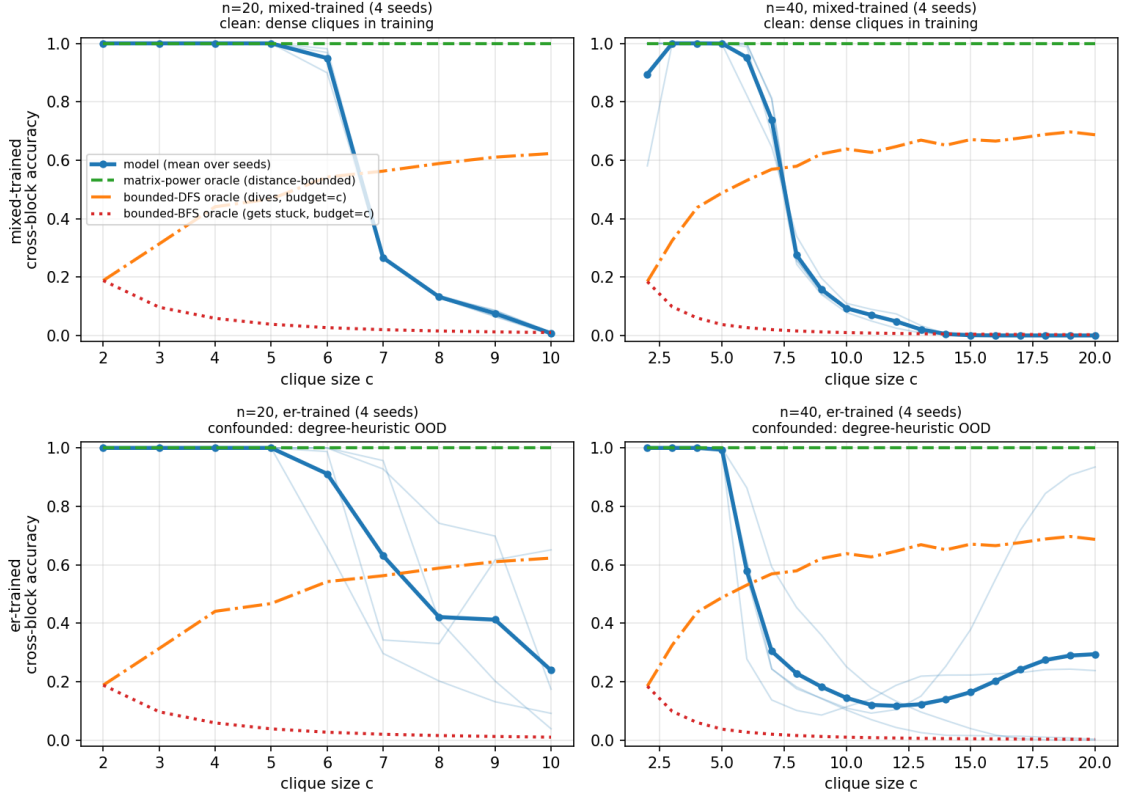


Figure 4: *Model*: the base connectivity transformer (RoBERTa trunk, linear read-out, $L = 2$, single head, $d_{\text{model}} = 512$, full-matrix $\hat{R}$), eval-only; *top row* **mixed**-trained (clean: dense cliques are in training), *bottom row* **ER**-trained (confounded: degree-heuristic OOD on dense cliques); $n = 20$ (left), $n = 40$ (right); four seeds each (thin lines = seeds, thick = mean). *Test set*: bridged cliques, ≥ 400 graphs per clique size. *Metric*: cross-block pair accuracy (does $\hat{R}$ connect the two cliques?) vs clique size c , against all three reference oracles. The bridge sits at distance ≤ 3 at *every* c , so the **matrix-power** oracle (green dashed) is flat at 1; the **bounded-DFS** oracle (orange dash-dot) *rises* with c (it dives across the bridge and reaches deeper into the far clique as the budget grows); the **bounded-BFS** oracle (red dotted) stays low (it gets stuck in the near clique). The mixed model (blue) collapses from 1 to 0 once the near clique exceeds ~ 6 –7 nodes — at roughly the same *absolute* clique size for both n — i.e. it follows the falling BFS branch and is the *opposite* of the rising DFS, ruling out the depth-first reading. The ER curves (bottom) fall too but are noisy and confounded (see text), and are shown only as a cross-check.

clique size c	2	3	4	5	6	7	8	10	14
model, $n = 20$ (mean)	1.00	1.00	1.00	1.00	0.95	0.27	0.13	0.01	—
model, $n = 40$ (mean)	0.89 [†]	1.00	1.00	1.00	0.95	0.74	0.27	0.09	0.00
matrix-power oracle	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

Table 5: *Model*: the **mixed**-trained base connectivity transformer (RoBERTa trunk, linear read-out, $L = 2$, single head, $d_{\text{model}} = 512$, full-matrix $\hat{R}$), eval-only, four seeds, $n = 20$ and $n = 40$. *Test set*: bridged cliques, ≥ 400 graphs per clique size. *Metric*: cross-block pair accuracy on the bridged graph, *mean over the four seeds*, by clique size c (selected columns; c runs to 10 at $n = 20$ and to 20 at $n = 40$, all 0.00 from $c = 14$ on). The matrix-power oracle stays flat at 1 because the bridge is always ≤ 3 hops; the model collapses with c . [†]the $n = 40$, $c = 2$ mean is pulled down by one noisy seed (0.58; the other three are ≈ 1.00).

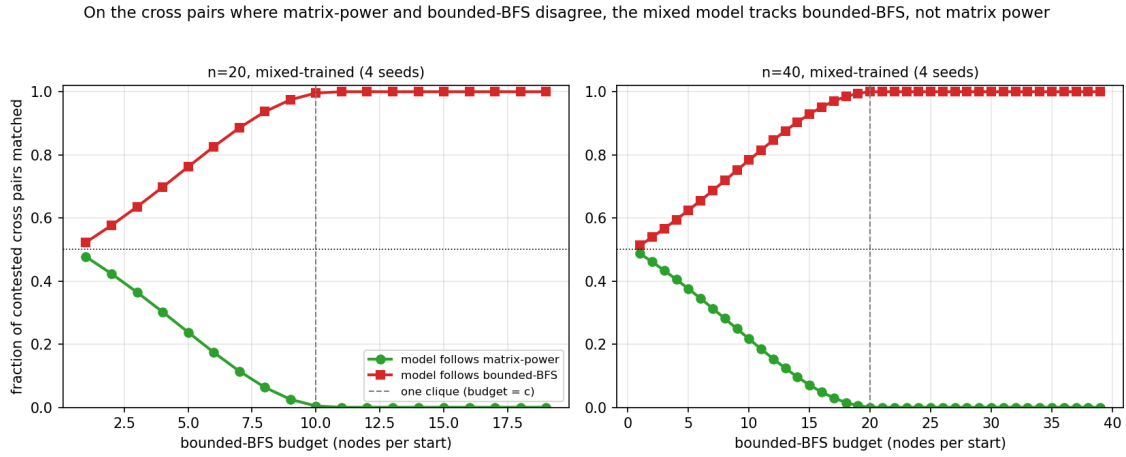


Figure 5: *Model*: the **mixed**-trained base connectivity transformer (RoBERTa trunk, linear read-out, $L = 2$, single head, $d_{\text{model}} = 512$, full-matrix $\hat{R}$), eval-only, four seeds; $n = 20$ (left), $n = 40$ (right). *Test set*: bridged cliques at full clique size, 500 graphs. *Metric*: restricted to the cross pairs where the matrix-power oracle and the visit-bounded-BFS oracle *disagree* (the bridge pairs), the fraction the model matches to each oracle, as the BFS budget b varies. The model tracks the early-stopping BFS, not matrix powering: as b approaches one clique (dashed line) the model matches BFS on $\rightarrow 1.0$ of the contested pairs and matrix powering on $\rightarrow 0$. This is the same fact as the cross collapse, read against the two reference algorithms.

n	seed	within-A	within-B	cross	discrimination
20	1000	1.00	1.00	0.09	0.50
20	2000	0.26	0.25	0.18	0.50
20	3000	0.62	0.61	0.04	0.50
20	4000	0.69	0.70	0.66	0.65
40	1000	0.30	0.30	0.24	0.50
40	2000	0.00	0.00	0.00	0.50
40	3000	0.96	0.96	0.93	0.50
40	4000	0.02	0.02	0.00	0.50

Table 6: *Model*: the **ER**-trained base connectivity transformer (RoBERTa trunk, linear read-out, $L = 2$, single head, $d_{\text{model}} = 512$, full-matrix $\hat{R}$) — the *confounded* cross-check — eval-only, four seeds, $n = 20$ and $n = 40$. *Test set*: bridged cliques at full clique size ($c = 10$ at $n = 20$, $c = 20$ at $n = 40$), 2,000 graphs. *Metric*: per-block pair accuracy on the bridged graph (within each clique, and the cross block), and the graph-level discrimination accuracy. Contrast with the clean mixed model (Table 4, where within-A and within-B are a flat 1.00): here the within-clique accuracy is all over the place (0.00 to 1.00) and swings wildly by seed, because the ER model has never seen a dense clique and is in the degree-heuristic OOD regime of Report IV — it is already wrong *inside* the cliques, so a low cross accuracy cannot be blamed on the bridge. Seed 3000 at $n = 40$ even gets a high cross accuracy (0.93) for the wrong reason (it calls everything dense “connected”), yet its discrimination stays at chance (0.50) because it then also calls the *split* cliques connected. This is why we read the conclusion off the mixed model, not this one.

4.3 Density in optimisation: denser training is faster, not slower

Setup recap. This is the training-side test of Section 3.3. We train the base connectivity model — the same RoBERTa-faithful trunk with the *linear* read-out ($L = 2$, single head, $d_{\text{model}} = 512$) — from scratch on a pure ER stream at $n = 20$, for 1,000,000 steps, four seeds, sweeping the edge probability $p \in \{0.05, 0.08, 0.12, 0.16, 0.22\}$ (mean degree $\approx 1.0, 1.6, 2.4, 3.2, 4.4$; the $n = 20$ connectivity threshold $\ln 20/20 \approx 0.15$ sits inside the sweep). Nothing else changes between runs. This experiment does *not* separate the two visit-bounded algorithms from each other — *both* bounded-DFS and bounded-BFS are bounded by the amount of structure to traverse, so both predict a *denser* stream should be *harder* / *slower* to fit. What it separates is the visit-bounded family as a whole from matrix powering, which predicts the *opposite* (denser graphs have shorter distances, so connectivity is easier). We read two things off each run: how fast and how reliably the in-distribution exact match converges, and how well the trained model transfers out of distribution.

Denser training converges faster and more reliably — the matrix-powering direction. Figure 6 and Table 7 are unambiguous: as the training density rises, the model reaches a given in-distribution accuracy in *fewer* steps, not more. The number of steps to 0.99 exact match drops from $\sim 10^5$ at $p = 0.05$ to $\sim 7 \cdot 10^4$ at $p = 0.12$ and to $\sim 5,000$ (the very first evaluation) at $p \geq 0.16$; the accuracy at the first eval (5,000 steps) climbs monotonically with p ($0.71 \rightarrow 0.57 \rightarrow 0.93 \rightarrow 0.99 \rightarrow 0.999$). Reliability moves the same way: at the sparse end some seeds never finish converging (one of four at $p = 0.05$, two of four at $p = 0.08$ plateau at ≈ 0.95 – 0.98 , never crossing 0.99 — the familiar seed lottery of Reports I–III), whereas every seed at $p \geq 0.12$ reaches exactly 1.00. So denser training is both faster and less of a coin-flip. This is the *matrix-powering*-consistent outcome and the opposite of what the strong “density hurts optimisation” form of *either* visit-bounded hypothesis (DFS or BFS) predicts.

The honest caveat: ER conflates density with distance, so this is not a clean probe of the traversal axis. Two things must be said plainly. First, the very sparse end is non-monotone: $p = 0.08$ is actually the *noisiest* setting (worst seed lottery), not $p = 0.05$, because near the connectivity threshold the target matrix R is a genuine mix of connected and disconnected pairs (hardest to fit), while at $p = 0.05$ it is almost all-zeros and at $p \geq 0.16$ almost all-ones (both trivial). Second, and more important: in an ER graph, raising the density *shortens distances* at the same time as it adds edges, and beyond $p \approx 0.16$ the in-distribution task is essentially trivial (the graph is one component, R is all-ones, the first eval is already at ceiling — the saturation we flagged in advance). So “denser = faster” here is largely the connectivity *task* getting easier (shorter distances, simpler target), *not* a clean measurement of the quantity a bounded traversal (DFS or BFS) is sensitive to. The ER sweep cannot isolate “more nodes to traverse at fixed distance,” because in ER more edges means *shorter* distance, the reverse coupling. That clean isolation is exactly what the bridged-clique experiments do (bridge fixed at ≤ 3 hops, only the clique size grows), and there — Sections 4.1–4.2 — the bounded-traversal cost *does* appear (the classifier’s transient slowdown with clique size, the base model’s cross-block collapse). The two results are consistent: when density and distance move together (ER), the distance effect wins and denser is easier; when they are decoupled (cliques), the traversal-cost effect surfaces. The ER sweep therefore rules out the strong claim that dense *training* is simply harder for *any* visit-bounded traversal, without touching the bounded-BFS evidence from the clique tests.

OOD transfer peaks just below the connectivity threshold. The more interesting read is the out-of-distribution transfer (Figure 7, Table 8). On the long, sparse structured families held out of this stream (1cycle, path-union, chain-plus), exact match is *non-monotone* in p , with a clear peak at $p = 0.12$ — just below the connectivity threshold — and a collapse on either side. Too sparse ($p = 0.05$ – 0.08): transfer is a seed lottery (only the one or two seeds that fully converge transfer at all; the rest are near 0). Too dense ($p \geq 0.16$): transfer collapses across all seeds, because a model trained on short, dense graphs never practises long-range reach, so it fails the long structured graphs — $p = 0.22$ gives ≈ 0 on every held-out family. The sweet spot at $p = 0.12$ is also the *least* seed-dependent (all four seeds agree: 1cycle = 1.00, path-union ≈ 0.73 , chain-plus ≈ 0.37), because at that density the training graphs span the widest useful range of distances. This is a small but clean “don’t train too dense” lesson, and it is consistent with the distance reading: dense training narrows the distances seen and costs the model its long-range reach.

p	seed	steps to 0.90	steps to 0.99	final exact
$p = 0.05$ (mean degree ≈ 1.0)				
0.05	1000	10k	105k	1.000
0.05	2000	25k	—	0.979
0.05	3000	10k	150k	0.994
0.05	4000	15k	210k	0.993
$p = 0.08$ (mean degree ≈ 1.6 ; paper’s density)				
0.08	1000	25k	75k	1.000
0.08	2000	55k	—	0.947
0.08	3000	40k	90k	1.000
0.08	4000	50k	—	0.947
$p = 0.12$ (mean degree ≈ 2.4)				
0.12	1000–4000	5k	70–75k	1.000
$p = 0.16$ (mean degree ≈ 3.2 , above threshold)				
0.16	1000–4000	5k	5–10k	1.000
$p = 0.22$ (mean degree ≈ 4.4 , well above threshold)				
0.22	1000–4000	5k	5k	1.000

Table 7: *Model*: the base connectivity transformer — RoBERTa-faithful trunk, *linear* read-out ($L = 2$, single head, $d_{\text{model}} = 512$) — trained from scratch on a pure **ER** stream at $n = 20$ for 1,000,000 steps, four seeds per density. *Training/eval set*: freshly-generated $\text{ER}(20, p)$ graphs (online stream); the metric is on a held-out $\text{ER}(20, p)$ validation batch at the same p . *Metric*: training steps to reach 0.90 and 0.99 **in-distribution exact match**, and the final exact match, *per seed* (the $p \geq 0.12$ rows collapse the four seeds, which agree to within one eval grid point of 5,000 steps). “—” marks seeds that never reach 0.99 (bold final < 1). Denser training converges in fewer steps and with fewer stragglers; the noisiest setting is $p = 0.08$ (near the connectivity threshold, where the target is hardest), not the sparsest. The first evaluation is at step 5,000, so “5k” means already converged at the first checkpoint.

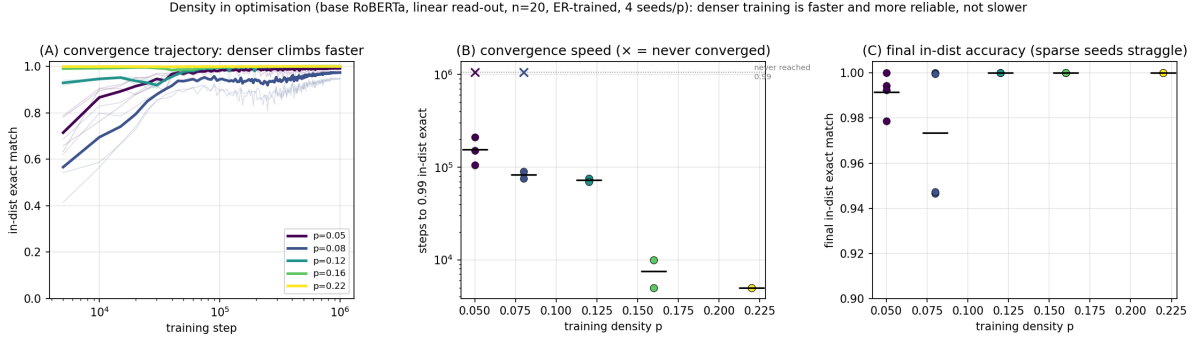


Figure 6: *Model*: the base connectivity transformer (RoBERTa-faithful trunk, *linear* read-out, $L = 2$, single head, $d_{\text{model}} = 512$), trained from scratch on a pure **ER**(20, p) stream, four seeds per density, 1,000,000 steps. *Eval set*: held-out $\text{ER}(20, p)$ at the same p . *Metric*: in-distribution exact match. **(A)** convergence trajectory vs training step (log axis; thin = each seed, thick = mean per density, colour = p): denser climbs faster. **(B)** steps to reach 0.99 exact vs p (per seed; black bar = mean over the converging seeds; “x” = a seed that never reached 0.99): monotone speed-up, by two orders of magnitude. **(C)** final exact match vs p (per seed): the sparse seeds straggle below 1.0, every seed at $p \geq 0.12$ is exactly 1.0. The speed-up is the matrix-powering direction (denser = shorter distances = easier), with the caveat that at $p \geq 0.16$ the task is near-trivial (see text).

family	$p=0.05$	$p=0.08$	$p=0.12$	$p=0.16$
1cycle	1.00 / 0 / 0 / 0	1.00 / 0 / 0.77 / 0	1.00 (all 4)	0.78 / 0.99 / 0.82 / 0.49
path-union	0.41 / .05 / .02 / .04	0.74 / .04 / .35 / .01	0.71–0.74	0.37 (all 4)
chain-plus	0.33 / 0 / 0 / 0	0.26 / 0 / 0.24 / 0	0.36–0.39	0.23 (all 4)

Table 8: *Model*: the base connectivity transformer (RoBERTa-faithful trunk, linear read-out, $L = 2$, single head, $d_{\text{model}} = 512$), **ER**(20, p)-trained, four seeds per density; *eval-only* on families held out of the ER stream. *Test set*: the long structured families of Report IV (1cycle, path-union, chain-plus), ≥ 400 graphs each. *Metric*: **OOD exact match**, listed *per seed* (seeds 1000/2000/3000/4000) where they disagree, or as a range/“all 4” where they agree; $p = 0.22$ omitted (all families ≈ 0 , every seed). Transfer is non-monotone and peaks at $p = 0.12$, just below the connectivity threshold, where it is also the most seed-consistent; too sparse is a seed lottery, too dense collapses (the model never sees long distances). This matches the distance reading of the convergence result.

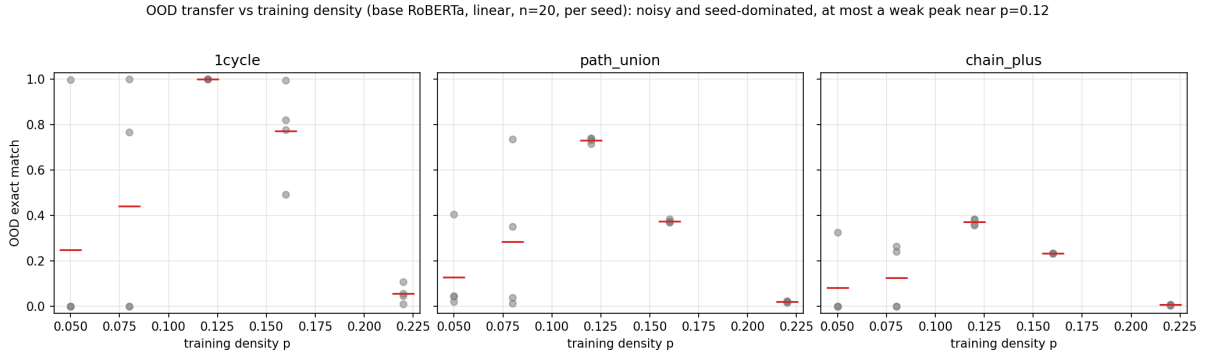


Figure 7: *Model*: the base connectivity transformer (RoBERTa-faithful trunk, linear read-out, $L = 2$, single head, $d_{\text{model}} = 512$), **ER**(20, p)-trained, four seeds per density; *eval-only*. *Test set*: long structured families held out of the ER stream (1cycle, path-union, chain-plus), ≥ 400 graphs each. *Metric*: OOD exact match vs training density p (grey dots = each seed, red bar = mean). Transfer is noisy and seed-dominated at the sparse end, peaks at $p = 0.12$ (and is cleanest there — all four seeds agree), and collapses for $p \geq 0.16$ as the dense training graphs stop exposing the model to long distances.

4.4 Oracle agreement across the natural families (capstone)

[Results pending — HPC job 533393 (oraclefam) still queued at 2026-06-19; pull runs/report5/oracle_families/, run plot_oracle_families.py, then write this as the last evidence subsection before the verdict.]

4.5 Does the node budget scale with the read-out?

[Results pending — launch scripts/eval_bridged_similarity.sbatch (eval-only, ~ 30 min), pull runs/report5/bridged_similarity/, extend plot_bridged_cliques.py, then report the knee location vs the linear baseline of Section 4.2.]

4.6 Capacity or data prior? Training on bridged cliques, and a depth sweep

[Results pending — launch scripts/train_bridged_in_stream.sbatch (Exp 2, decisive) and optionally scripts/train_bridged_depth_sweep.sbatch (Exp 1, bonus). Compare the cross-block-vs-clique-size curve to the held-out baseline of Section 4.2: same collapse = capacity, no collapse = data gap; knee moving with $L =$ the 3^L capacity.]

5 Verdict: what is the model computing?

[To be written once Sections ??–?? land.]